

## REVUE CONTRADICTOIRE · PROPOSITION « DUAL-ROLE DUAL-FLOW »

# Le principe tient. Trois piliers du document, non.

Confronté au schéma Prisma, aux 86 routes et aux deux audits précédents — positionnement des pages et modèle de sécurité. Chaque verdict ci-dessous est adossé à une ligne de code réelle, pas à une lecture du document seul.

**VERDICT**

Le **concept** de compte unique à deux visages est non seulement pertinent, il est **déjà à moitié construit** dans Flexwork\_ios — l'audit de routage l'avait montré sans le nommer. Mais le **document** est un plan de marketplace générique de marché français : il propose un portefeuille interne, un mot de passe et deux tables de profils séparées, qui contredisent chacun une décision structurante déjà prise ici. Et il classe l'auto-attribution en « risque à anticiper » alors que la faille est **ouverte aujourd'hui**.

4 points alignés

3 à adapter

4 contradictions

## Ce que le document voit juste

Et surtout : ce sur quoi le projet a déjà pris de l'avance sans que le document le sache. L'audit de routage avait identifié comme la différence n°1 entre les deux dépôts le fait que Flexwork\_ios range par *cycle de vie* et non par *propriétaire*. C'est précisément ce qui rend le dual-role possible ici, et coûteux ailleurs.

**L'espace mission est déjà neutre, donc déjà bi-rôle**

ACQUIS

Toutes les étapes du cycle — devis, candidatures, contrat, séquestre, livrable, pointage, médiation, avis — vivent sous /missions/[id]/\*, partagé par le client et le prestataire.

Aucune route n'est dupliquee par role. Un utilisateur qui change de casquette ne change pas d'URL : il voit la même page sous un autre angle.

### CONTRE-EXEMPLE

*flexwork\_job imbrique la mission sous son client*

(*dashboard/client/missions/[id]/contract/[contractId]*). Y ajouter le *dual-role* exigerait de reconstruire l'arborescence entière.

**PREUVE** Audit de routage, constat n°1 – 9 façades sous missions/[id]

## Le multi-profil existe déjà, et il est plus souple que proposé

## ACQUIS

Le document propose « 0, 1 ou 2 profils ». Le schéma permet déjà **N profils nommés** par utilisateur – « Maçon local », « Conseil BTP à distance » – chacun avec son domaine, ses tarifs, sa zone d'intervention, ses garants et ses déclarations. La règle du document est donc un sous-ensemble de ce qui tourne déjà.

```
PREUVE schema.prisma:92      profiles      - schema.prisma:316  @@unique([userId,
, Profile[]               , label])
```

## Le KYC est porté par le compte, pas par le rôle

## ACQUIS

User.kycStatus est unique par personne. Une identité vérifiée une fois vaut pour les deux faces – exactement le comportement attendu en dual-role, et un point que le document ne traite pas du tout alors qu'il aurait pu être un point de friction majeur.

```
PREUVE schema.prisma:75 · kycStatus KycStatus @default(en_attente) sur User
```

## Les deux faces sont déjà distinguées en base

## ACQUIS

User.missionsClient et User.proposalsSoumises sont deux relations distinctes sur la même personne. La base sait déjà dire « ce que je publie » et « ce à quoi je candidate ». Le « mode » du document n'a donc pas besoin d'être stocké : il est **dérivable**.

**PREUVE** schema.prisma:94-95 · relations MissionClient et ProposalProvider

## Les quatre contradictions

Chacune oppose une proposition du document à une décision déjà arbitrée dans ce projet — parfois pour des raisons juridiques documentées dans le schéma lui-même. Ce ne sont pas des divergences de goût.

## Le portefeuille unifié réintroduit ce qui a été supprimé pour raison légale

CONTRADICTION

### LE DOCUMENT PROPOSE

« Deux wallets logiques sous un même compte Stripe Connect » · « Le wallet est unifié mais les flux comptables sont séparés » · « Portefeuille : vue unifiée avec solde global ».

Le projet a explicitement **éliminé tout solde interne**. Le modèle en vigueur est : la plateforme *instruit* un prestataire de paiement agréé, elle ne détient jamais les fonds et n'apparaît jamais comme bénéficiaire. Il n'y a ni Wallet, ni WalletTransaction, ni Retrait — uniquement des PspEscrowOperation, c'est-à-dire des instructions.

Adopter le portefeuille du document ne serait pas une évolution : ce serait revenir sur l'arbitrage qui fonde la position juridique de la plateforme, et rouvrir un prérequis déjà classé *bloquant et non traité*.

**PREUVE** schema.prisma:790-791 · « ne détient jamais les fonds · Remplace intégralement TransactionEscrow/Wallet/WalletTransaction/Retrait — aucun solde interne n'existe plus »

## Le document est écrit pour le marché français

CONTRADICTION

siret VARCHAR(14), Stripe, montants implicitement en euros. Le projet opère au Bénin et en Afrique de l'Ouest : devise XOF par défaut partout, pays BJ / CI / SN / TG, paiements MTN MoMo, Wave et Orange Money, seuils d'âge administrables *par pays*. Le SIRET n'a pas d'existence dans ce cadre.

Ce n'est pas fatal — c'est un signal : le document n'a pas été écrit *pour ce projet*, et ses recommandations doivent être relues une par une plutôt qu'appliquées.

**PREUVE** schema.prisma:441,569,803 currency String · CountryAgeRequirement.country @default("XOF")

## Le mot de passe et la table de sessions contredisent l'authentification en place

CONTRADICTION

**LE DOCUMENT PROPOSE**

*password\_hash VARCHAR(255) · une table user\_sessions portant active\_role · un parcours « Email + mot de passe ».*

Il n'y a pas de mot de passe dans Flexwork\_ios : l'authentification se fait par code à usage unique. User.passwordHash existe mais n'est jamais renseigné. Et la session est un JWT sans état, revalidé en base à chaque requête — il n'y a pas de table de sessions à laquelle accrocher un rôle actif.

Conséquence concrète : le « rôle actif » ne peut pas se stocker là où le document le range. S'il devait exister, il irait dans le jeton — et toute bascule devrait forcer sa réémission, sans quoi l'utilisateur garderait l'ancien rôle jusqu'à expiration.

---

**PREUVE** src/auth.ts · provider Credentials id:"otp" , session: { strategy: "jwt" }

**Deux tables de profils défont une unification déjà faite****CONTRADICTION**

Le document sépare client\_profiles et freelance\_profiles. Le projet avait précisément *fusionné* quatre tables de profils métier en un modèle Profile générique, pour cesser de dupliquer la même structure par catégorie. Re-scinder en deux, c'est refaire le chemin à l'envers.

Par ailleurs le côté client n'a besoin d'aucun profil : les données que le document y range (adresse, localisation, identité) sont déjà sur User, partagées par les deux faces — ce qui est le comportement correct.

---

**PREUVE** schema.prisma:289-291 · « Profil générique unique (remplace ArtisanProfile/ExpertProfile/ManoeuvreProfile/DigitalProfile) »

## Le croisement que ni le document ni l'audit de sécurité ne pouvaient voir seuls

C'est le point le plus important de cette revue. Il n'apparaît qu'en tenant les deux documents ensemble, et il inverse l'ordre des travaux.

**Le dual-role invalide le garde de**

## sécurité proposé dans l'audit précédent

L'audit de sécurité recommandait un garde unique `requireMissionParty()` pour fermer la faille F-01, en autorisant sur la base d'une relation à la mission :

```
const mission = await prisma.mission.findFirst({
  where: { id: missionId, OR: [
    { clientId: userId }, // je suis le client
    { proposals: { some: { providerId: userId } } }, // je suis un candidat
  ] },
});
return { ok: true, mission, isOwner: mission.clientId === userId };
```

Sous un modèle mono-rôle, ces deux branches sont mutuellement exclusives : personne ne peut être les deux sur la même mission. **Sous dual-role, un même utilisateur peut satisfaire les deux.** Le garde l'autorise alors doublement, et `isOwner` – qui décide qui voit les offres concurrentes et qui peut accepter – devient ambigu au lieu d'être faux.

Autrement dit : le correctif de sécurité et l'évolution produit se contredisent silencieusement si on les traite dans le mauvais ordre.

## L'auto-candidature n'est pas un risque futur : elle est ouverte maintenant

FAILLE ACTIVE

LE DOCUMENT CLASSE EN « FRICTION À ANTICIPER »

« Un freelance ne peut pas candidater à sa propre mission. Un client ne peut pas acheter son propre Gig. »

Recherche exhaustive dans `src/app/api` et `src/lib` : **aucune comparaison entre le client d'une mission et l'auteur d'une candidature n'existe.** La route de candidature vérifie le KYC, le statut de la mission, les garants, l'assurance, les doublons et les rounds – jamais que l'appelant n'est pas le commanditaire.

Aujourd'hui la faille est masquée par l'interface : un compte de rôle client ne voit jamais le bouton « Candidater ». Mais l'API l'accepte, et le rôle n'y est pas contrôlé non plus. **Le dual-role transformerait une manipulation d'API en parcours utilisateur normal, à un clic.** Un commanditaire pourrait gonfler artificiellement le nombre de candidatures reçues, s'auto-attribuer la mission, et déclencher un contrat entre lui et lui-même.

**PREUVE** POST `/api/missions/[id]/proposals` · aucune occurrence de `clientId !== providerId` dans tout le dépôt

## La faille F-01 change de gravité

AGGRAVATION

L'audit de sécurité notait que `GET /api/missions/[id]/proposals` laisse tout compte connecté lire les offres concurrentes d'une mission quelconque. En mono-rôle, seuls les comptes prestataires ont un intérêt à exploiter cette fuite. **En dual-role, tout le monde est potentiellement concurrent** – la population exploitante passe d'un sous-ensemble à la totalité des inscrits.

## La barrière d'âge repose sur un rôle unique

À ARBITRER

Le contrôle d'âge des filières chantier – seul contrôle *effectif* de la plateforme, avec le KYC – se déclenche sur `isChantierRole(user.role)`. Si une même personne est cliente *et* manœuvre, quel rôle gouverne l'activation de son profil ? Le document ne pose pas la question, parce qu'une marketplace générique n'a pas de métier réglementé.

Réponse à trancher explicitement avant toute bascule : le contrôle doit porter sur le **profil prestataire**, jamais sur le compte – sans quoi un compte à double face échapperait au seuil légal, ou en subirait un sans objet côté client.

---

**PREUVE** `src/lib/age-gate.ts · isChantierProfileAllowed({ role, dateNaissance, minimumAge })`

# Le modèle adapté à ce projet

Beaucoup plus léger que le document, parce qu'il s'appuie sur ce qui existe. La différence tient en une phrase : **la face n'est pas un attribut du compte, c'est un attribut de la relation à une mission.**

## Ne pas stocker de « rôle actif ». Le dériver.

RECOMMANDATION

Le document veut un `active_role` en base et en session. Inutile ici : sur une mission donnée, je suis le client si `mission.clientId === moi`, et candidat si j'ai une proposition dessus. La base répond déjà. Le « mode » du document devient un simple filtre d'affichage, pas un état à synchroniser entre `localStorage`, table de sessions et JWT – trois sources de vérité que le document accepte, et qui divergeront.

`UserRole` reste alors ce qu'il est déjà : une **filière métier** (artisan, expert digital...) qui décrit

ce que je sais faire — pas de quel côté du marché je me trouve.

## Bloquer l'auto-attribution à l'écriture, pas à l'affichage

RECOMMANDATION

Deux lignes, à poser *avant* toute ouverture du dual-role — et de toute façon nécessaires aujourd'hui :

```
// POST /api/missions/[id]/proposals — a ajouter
if (mission.clientId === providerId) {
  return NextResponse.json({ error: "self_dealing_forbidden" }, { status: 403 })
}

// requireMissionParty — desambiguer les deux branches
const isOwner = mission.clientId === userId;
const isProvider = mission.proposals.some(p => p.providerId === userId);
if (isOwner && isProvider) throw new Error("invariant: self-dealing en base");
```

La seconde n'est pas défensive par excès : si les deux sont vrais, une donnée invalide existe déjà en base et aucune décision d'autorisation ne peut être prise sainement dessus.

## Séparer les notes par face — sans toucher au schéma

RECOMMANDATION

Le document propose deux colonnes `rating_client` et `rating_freelance`. Inutile : Review porte déjà `missionId`, `authorId` et `targetId`. La face de l'évalué se déduit de la mission — noté comme client si la mission était la sienne, comme prestataire sinon. Deux agrégats calculés, zéro migration, et aucun risque de désynchronisation entre une colonne dénormalisée et les avis réels.

---

**PREUVE** `schema.prisma:638-653 · Review avec @@unique([missionId, authorId])`

## Le portefeuille reste une vue, jamais un solde

RECOMMANDATION

L'écran « Wallet » peut parfaitement afficher « encaissé » et « dépensé » côte à côte — c'est une **agrégation de lecture** sur les opérations PSP confirmées. Ce qu'il ne doit jamais devenir, c'est un solde détenu par la plateforme. La distinction est invisible à l'écran et déterminante juridiquement.

# Ordre des travaux

L'ordre compte plus que le contenu : ouvrir le dual-role avant de fermer l'auto-attribution transformerait une faille théorique en fonctionnalité.

|    |                                                                                                                                     |          |
|----|-------------------------------------------------------------------------------------------------------------------------------------|----------|
| 01 | <b>Bloquer l'auto-candidature</b><br>Nécessaire indépendamment du dual-role – la faille est ouverte aujourd'hui.                    | BLOQUANT |
| 02 | <b>Fermer F-01, F-02, F-03</b><br>Le dual-role élargit la population exploitante de F-01 à tous les inscrits.                       | BLOQUANT |
| 03 | <b>Trancher la barrière d'âge A13</b><br>Le contrôle porte sur le profil prestataire, pas sur le compte. Décision à écrire.         | BLOQUANT |
| 04 | <b>Dériver la face au lieu de la stocker</b><br>Aucune migration : filtre de lecture sur les relations existantes.                  | ENSUITE  |
| 05 | <b>Unifier la navigation des deux faces</b><br>Réconcilier /client/* et /dashboard/<filière>/*, aujourd'hui deux espaces disjoints. | ENSUITE  |
| 06 | <b>Notes et historique par face</b><br>Agrégats calculés, sans dénormalisation.                                                     | ENSUITE  |

# Synthèse des verdicts

| PROPOSITION DU DOCUMENT | VERDICT | MOTIF |
|-------------------------|---------|-------|
|-------------------------|---------|-------|